Projet - Annuaire Symfony

1. Sujet - Annuaire en ligne

Ce projet se fera en **trinôme** et s'intéressera au développement d'une application **d'annuaire en ligne**.

L'objectif est de développer un site web classique en "server-side rendering" (qui gère à la fois la partie client et serveur) en utilisant Symfony et Twig (TD1, TD2 et TD3 de Symfony).

Voici les détails du service qui devra être développé :

- Le site web est un annuaire en ligne qui permet à chaque utilisateur de créer et de compléter un profil avec un certain nombre de champs informatifs.
- Le profil de l'utilisateur possède un mode de visibilité qui indique s'il peut être publiquement listé ou non. Un profil peut donc être soit **visible** (listé) ou bien **masqué** (non listé).
- La page principale du site doit afficher tous les profils **visibles**. À partir de cette page, on doit aussi pouvoir accéder facilement aux pages de profils des utilisateurs listés.
- Attention, même si le profil est **masqué**, il peut toujours être consulté via l'adresse et le code du profil. S'il est **masqué**, il n'est simplement pas listé sur la page principale.
- Lors de l'inscription (via un formulaire) l'utilisateur précise seulement un minimum d'informations : login, adresse email, mot de passe et la visibilité du profil (visible/masqué).
- Chaque profil doit être associé à un **code unique**. Pendant l'inscription, l'utilisateur peut choisir de préciser lui-même ce code ou non (à condition qu'il ne soit pas déjà pris). S'il ne précise rien, un code aléatoire sera alors généré. Quand l'utilisateur décide lui-même de saisir un code, l'application doit vérifier en temps réel que le code n'est pas déjà pris, avant la soumission du formulaire (donc, en utilisant du **javascript** et des **requêtes asynchrones**). Le code ne doit contenir que des caractères alphanumériques.
- Seul le **javascript** "nature" est autorisé (pas de framework JS, juste de simples fichiers **javascript**, comme dans le TD3).
- Une fois connecté, l'utilisateur peut **éditer son profil** avec des **informations complémentaires** (par exemple, numéro de téléphone, pays, adresse postale, réseaux sociaux, etc.). À vous de trouver les données qui vous semblent intéressantes à préciser sur le profil.
- Le **formulaire d'édition** du profil doit être automatiquement **pré-rempli**.
- À tout moment, l'utilisateur peut **changer le code associé à son profil** (soit en spécifiant un nouveau, soit en demandant la génération d'un code aléatoire).
- L'utilisateur peut **supprimer son profil**.
- Une route incluant le code du profil permet d'accéder et de visualiser la page de profil d'un utilisateur (par exemple `/profil/{code}`). Il n'y a pas besoin d'être connecté pour cela.
- L'utilisateur peut **changer la visibilité de son profil** (de **visible** à **masqué** ou inversement).
- En plus de la route qui permet de visualiser le profil de l'utilisateur sur une page dédiée, une autre route (qui inclue donc aussi le code secret du profil) doit renvoyer les informations de l'utilisateur au format `JSON` (donc, pas une page web complète, seulement les données). Cela vous servira plus tard, lors du 3e projet où vous utiliserez directement de ce service.
- Sur le profil, l'application doit afficher **la dernière date où a été édité le profil** et la **dernière date de connexion** de l'utilisateur.
- **Attention**, vous devrez faire en sorte que **la dernière date d'édition du profil** soit mise à jour dès que **l'objet** (entité) stockant l'utilisateur est mise à jour, peu importe l'endroit où cela est fait : dans un contrôleur, dans un service, dans une commande, etc. Il faut ainsi faire en sorte de ne pas avoir à dupliquer le code gérant cette logique si une nouvelle portion de code mettant à jour cette entité est implémentée.
- Le site doit pouvoir être passé en **mode maintenance** à l'aide d'un nouveau paramètre que vous pourrez définir et modifier dans le fichier `services.yaml`. Quand le site est en mode maintenance, toutes les pages du site doivent rediriger sur une page qui affiche un message expliquant que le site est actuellement en maintenance.

- Certains utilisateurs peuvent posséder le rôle d'**administrateur**. Sur la page principale du site, en plus des profils **visibles**, un administrateur peut aussi visualiser et accéder aux profils **masqués**. Aussi, à partir d'un profil, un administrateur peut **supprimer le compte** de l'utilisateur qui possède ce profil, sauf si cet utilisateur est aussi un administrateur.

- Une commande (**Symfony**) doit permettre de créer un utilisateur depuis le terminal en précisant ses informations et son rôle (normal/administrateur). Les informations pourront être données directement en argument de la commande, ou alors en mode interactif.

2. Aide et pistes

Pour vous aider dans la réalisation du projet, voici quelques pistes :

- Pour **modifier** un objet (entité) déjà existant (par exemple, un utilisateur), on récupère simplement l'objet correspondant et on applique les modifications (par exemple, avec ses **setters**). Ensuite, on utilise là-aussi le service `EntityManager` afin de synchroniser les modifications avec la base de données en utilisant la méthode `flush`. Plus d'information à ce propos sur la documentation officielle.

- Dans le TD1, nous avons utilisé l'attribut `#[ORM\PrePersist]` afin de créer la date de publication d'un message automatiquement juste avant l'enregistrement en base de données. D'autres attributs similaires peuvent vous permettre d'implémenter la fonctionnalité relative à la **date de dernière édition du profil**.

- L'attribut `#[ORM\PrePersist]` n'est utilisé que dans le cas d'une insertion dans la base de données (donc, quand l'entité est créée, pas lorsqu'elle est mise à jour).

- Lors des TDs PHP de deuxième année, vous avez vu comment générer une chaîne aléatoire (par exemple ici, au niveau de la section **1.2**).

- Dans les TD2, nous avons vu comment capter des **événements** (`LoginSuccessEvent`, `LoginFailureEvent` et `LogoutEvent`) afin d'exécuter du code lorsque ces événements surviennent. D'autres événements peuvent vous permettre de facilement implémenter la fonctionnalité relative à **la date de dernière connexion** d'un utilisateur ainsi que le **système de maintenance**.

- Pour réaliser la fonctionnalité qui vérifie l'unicité du code du profil pendant l'inscription, à l'aide des requêtes asynchrones, vous pouvez vous inspirer de ce TD de javascript de 2e année, notamment pour mettre en place un système de **debouncing**.

- Dans le TD3, nous avons vu qu'il faut renvoyer un objet `JsonResponse` quand on veut renvoyer une réponse au format JSON plutôt qu'une page web complète (générée avec **twig**). Vous trouverez plus d'informations ici.

- La création de commandes et la gestion des rôles sont également abordées lors du TD3.

- Pour le style du site, faites ce que vous voulez, tant que ce n'est pas trop laid ! Cependant, **il est interdit de reprendre le style de The Feed**. Par contre, vous pouvez utiliser n'importe quel Framework CSS (par exemple bootstrap, tailwind css ou bien quelque chose d'encore plus simple comme bulma). De plus, **Symfony** vous permet d'intégrer facilement bootstrap pour générer facilement des formulaires stylisés. Le site de tailwind css possède aussi un guide d'installation pour Symfony.

3. Rendu

La **deadline** du projet est le **samedi 28 septembre 2024, 23h59**.

Le projet sera à rendre sur **Moodle**. Un seul membre du trinôme dépose une archive **zip** nommée selon le format : `NomPrenomMembre1-NomPrenomMembre2-NomPrenomMembre3.zip`.

Cette archive devra contenir :

- Les sources de votre projet. Attention à ne pas inclure les répertoires **vendor** et **var**.

- Un fichier **README** qui contient :

    - Le lien du dépôt git où le code source de l'application est hébergé.

    - Le fonctionnement de l'annuaire : comment utiliser l'application comment accéder à un profil, les différentes routes, comment utiliser la commande pour créer des utilisateurs, etc.

    - Un récapitulatif de l'investissement de chaque membre du groupe dans le projet (globalement, qui a fait quoi).

- Éventuellement, des indications supplémentaires s'il y a des choses particulières à faire pour lancer et tester votre application en local (autrement que de lancer le serveur, configurer et générer la base de données, etc...)

4. Déroulement du projet et accompagnement

Globalement, la plupart des fonctionnalités sont réalisables à partir des connaissances que vous avez acquises pendant ce cours. Parfois, il faudra un peu adapter. Par exemple, nous n'avons pas fait de fonctionnalité "modifier le profil" sur le site de base de The Feed, mais vous devriez être capable d'implémenter une telle fonctionnalité sur votre projet. Aussi, il faudra vous référer à la section "Aide et pistes" de cette page pour pouvoir implémenter certaines fonctionnalités.

Il faudra aussi faire des recherches et vous documenter par vous-même. Du côté de Symfony, vous pouvez notamment consulter la documentation officielle, mais vous trouverez également de l'aide sur de nombreux forums. Vous pouvez même chatter directement sur le Slack de Symfony.

Bien entendu, vous pouvez utiliser des outils de gestion de projet appropriés (Trello par exemple ?).

Il faudra aussi utiliser et bien organiser un dépôt git. N'oubliez pas que vous pouvez utiliser le Gitlab du département.

N'hésitez pas à poser des questions à votre enseignant chargé de TD et à montrer votre avancement ! Bon projet.

(cc) BY-SA Malo Gasquet, Romain Lebreton, Cyrille Nadal et Matthieu Rosenfeld 2024, licensed under CC-BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0/>.